

AI Image & Video Compressor

Complete Guide: Basic to Advanced - How It Works, The Maths, AI-Adaptive vs Fixed Baseline, Interview Preparation & Edge Cases

Covers the actual codebase: analyzer, selector, compressor, evaluator, scoring, pipeline, FastAPI backend, static frontend, benchmark harness. Everything is honest about what is implemented vs what is roadmap.

Stack: Python, Pillow, numpy, ffmpeg (H.264), FastAPI. Metrics: PSNR, SSIM, 1-10 composite score. DB: sqlite jobs log.

1. Big Picture (30-Second Version)

Upload an image or video. The backend decides image vs video by file extension. In AI mode the file is analyzed, classified into a content class, and given custom compression settings. It is compressed, then real quality (PSNR/SSIM) is measured. If quality or size targets are missed, settings are auto-adjusted and it re-encodes (up to 4 attempts). Every result is scored 1-10 against a fixed baseline (JPEG q75 / H.264 CRF23) so the AI must prove it wins.

```
browser -> POST /api/compress -> compress_auto() -> analyze -> predict  
-> compress -> evaluate -> retry? -> score -> sqlite log -> download URL
```

2. Images: Basics to Maths

2.1 What an image is

A digital image is a grid of pixels. Each pixel in RGB is 3 numbers (Red, Green, Blue), 0-255 each. A 1600x1200 photo = 1.92M pixels x 3 bytes = ~5.5 MB raw. Compression removes redundancy so we store far less.

2.2 Why images compress: three redundancies

- **Spatial:** neighbours look alike (blue sky). Predict a pixel from neighbours, store only the difference.
- **Perceptual:** eyes notice brightness detail more than colour detail - so JPEG stores colour at half resolution (chroma subsampling 4:2:0).
- **Statistical:** common patterns get short codes (Huffman / entropy coding).

2.3 JPEG maths (the fixed baseline)

Steps: (1) RGB to YCbCr, downsample Cb/Cr. (2) Split into 8x8 blocks. (3) DCT - converts each block into 64 frequencies (like a prism splitting light). (4) Quantization - divide high frequencies by large numbers and round: this is the LOSSY step, controlled by the quality parameter q. (5) Zigzag + run-length + Huffman coding (lossless). Lower q = coarser rounding = smaller file, more block artifacts.

2.4 WebP maths (the AI default)

WebP lossy reuses VP8 video intra-frame coding: predict each block from already-decoded neighbours (many directional modes), store only the residual, transform + quantize it, entropy-code with LZ77 + Huffman. Better prediction than JPEG's DCT-only approach is why WebP is typically 25-35% smaller than JPEG at equal quality. The quality parameter plays the same role: quantizer step size.

2.5 PSNR and SSIM (how we judge, with formulas)

MSE = mean of squared pixel differences. PSNR = $20 \times \log_{10}(255 / \sqrt{\text{MSE}})$, in dB. +6 dB means roughly 4x less error; our floor ~32-33 dB. PSNR is pure signal math - blind to perception.

SSIM compares luminance, contrast and structure in windows: $SSIM = l \times c \times s$, each 0-1, using local means, variances and covariance with stabilizers $C1=(0.01 \times 255)^2$, $C2=(0.03 \times 255)^2$ (see `src/evaluator.py:ssim`). SSIM 0.95 $\sim$ '95% information retained' - our headline target. SSIM 1.0 = identical.

2.6 Our image pipeline (code mapping)

Stage	File:function	What happens
Analyze	<code>analyzer.py:features()</code>	256px gray copy: Laplacian-variance texture, gradient edge density, colorfulness, 32-bin luminance entropy, megapixels
Classify	<code>analyzer.py:classify()</code>	graphic-screen / smooth-lowdetail / textured / photo-natural + confidence
Predict	<code>selector.py:predict_image_params()</code>	class -> WebP quality 70-85; scale 0.85 if >4MP, 0.75 if >12MP
Compress	<code>compressor.py:compress_image()</code>	Pillow save with quality/scale
Evaluate	<code>evaluator.py:image_metrics()</code>	real PSNR + SSIM on ≤ 512 px gray copies
Retry	<code>pipeline.py</code> loop + <code>selector.adjust_on_quality()</code>	ssim quality+8; saving low with headroom -> quality-8; max 4 tries

3. Video: Capture to Compressed File

3.1 What video is

Video = 25-60 still images (frames) per second + usually an audio track, inside a container (MP4). Raw 1080p30 = 1920x1080x3 bytes x30 = ~ 187 MB/s. Compression exploits a 4th redundancy on top of image ones:

- **Temporal redundancy:** consecutive frames are nearly identical. Store one full frame, then only what changed.
- **I-frame (keyframe):** full image, like JPEG. **P-frame:** 'take previous frame, move these blocks here (motion vectors), add this small correction'. Skipped blocks cost $\sim$ nothing.
- **Motion estimation:** encoder searches previous frame for matching 16x16/8x8 blocks - the most expensive step (this is why presets matter).
- **GOP:** group of pictures between I-frames (e.g. 250 frames). More P-frames = smaller, but seeking/errors need the next I-frame.

3.2 H.264 encode pipeline (what ffmpeg does for us)

```
frames -> motion estimation (vectors) -> residual = frame - prediction
-> 4x4/8x8 integer transform -> quantization (CRF controls step)
-> entropy coding (CABAC) -> MP4 | audio -> AAC 128k
```

CRF (Constant Rate Factor, 0-51, we use 18-33) sets quality: +2 CRF $\sim$ 25% fewer bits at slightly lower quality. Preset (ultrafast..veryslow) trades encode time for compression efficiency at the same CRF: slower = smarter motion search. Decode is the exact inverse: parse -> dequantize -> inverse transform -> add motion prediction -> display. Decoding is cheap; encoding is expensive.

3.3 Our video pipeline (code mapping)

- Classify from a **real extracted frame** at $t=1s$ (`_video_thumbnail_features`) - never fabricated numbers.
- Map class to CRF 20-28, preset veryfast, optional downscale (`selector.py:predict_video_params`).
- Encode: `ffmpeg libx264 + AAC` (`compressor.py:compress_video`).
- Measure **real** quality: extract frames at $t=0.5/1.5/2.5s$ from source AND output, compare PSNR/SSIM (`evaluator.py:video_metrics`).

- Retry: CRF-3 if quality low, CRF+2 if saving low with headroom.

Honest limit: frame-sample SSIM is a proxy, not full VMAF. VMAF-guided quality is roadmap (see RESEARCH.md).

4. AI-Adaptive vs Fixed Baseline

Purpose of fixed: a frozen reference (image JPEG q75; video H.264 CRF23 preset medium, single pass, never retried) so the AI must prove itself on identical inputs. Purpose of AI: different content needs different settings - smooth graphics compress far more than noisy texture at the same perceived quality, so one setting for all files wastes bytes or damages detail.

Aspect	Fixed baseline	AI-adaptive
Decided in	pipeline.py else-branch (params hardcoded)	analyzer + selector (params predicted per file)
Attempts	exactly 1 (rounds=1)	up to 4 (max_retries+1) with measured retry
Quality check	none - output accepted blind	real PSNR/SSIM every attempt, floor min_ssim
Measured photo.jpg	7233 B, SSIM 0.9999, 9.14/10	1632 B, SSIM 0.9999, 9.79/10
Measured graphic.png	4188 B - BIGGER than input (-158%)	924 B (WebP switch), 8.56/10
Measured clip.mp4	36834 B, 8.07/10	25807 B, 8.65/10

Read the wart honestly in interviews: fixed JPEG inflating a tiny PNG proves the point - blind settings fail on some content, adaptation wins.

5. How Much AI / ML / Deep Learning / CNN Is Used?

Honest accounting - say exactly this in interviews:

Technique	Used?	Where / notes
Hand-crafted features + statistics	YES (core)	texture/edge/color/entropy in analyzer.py
Rule-based classifier (heuristic policy)	YES (core)	classify() thresholds; the 'AI brain' of v1
Closed-loop control (measure -> adjust -> retry)	YES (core)	pipeline retry loop; the intelligence that guarantees floors
Classical ML (sklearn trees/SVM)	NO	upgrade path: train classifier on labeled DIV2K/CLIC crops
CNN / deep learning at runtime	NO	0% - no neural net runs in v1; deliberate CPU/ms-latency choice
Learned codecs (ELIC/MLIC, DCVC, JPEG-AI)	NO (surveyed)	docs/RESEARCH.md: beat VVC ~8-24% BD-rate but need GPU + custom decoders
Bayesian optimization	NO (roadmap)	search quality/CRF space instead of fixed +-steps

Defensible line: v1 is an adaptive decision system with measured feedback that beats the baseline on every sample - ML in the 'learning from data to make decisions' sense, with a documented path to learned models. Never claim a CNN runs in v1.

6. Scoring Maths (src/scoring.py)

score = 4.0xSSIM + 2.5xPSNRnorm + 2.5xsaving + 1.0xspeed, clipped 0-10. PSNRnorm=(PSNR-20)/20. Speed = 1/(1+t/10) images, 1/(1+t/120) video. Verdicts: SSIM>=0.95 & saving>=60% = ACCEPT (excellent); >=0.90 & >=50% = ACCEPT (good); >=0.85 = MARGINAL; else REJECT; unevaluable quality = NEEDS-REVIEW. Acceptance bar 7.0.

7. Interview Q&A; Bank

Question	Answer (30-45s)
Walk me through a request	Browser POSTs file+mode; main.py saves it, picks webp/mp4 by extension, calls compress_auto; pipeline analyzes, classifies, predicts, encodes, measures, retries; result logged to sqlite, JSON + download URL returned.
Why not just fixed q75?	One setting misfires per content: our fixed JPEG inflated a PNG by 158%. Adaptation picks per-file settings and verifies.
Why WebP default?	~25-35% smaller than JPEG at equal quality via better VP8 intra prediction; JPEG kept as fixed baseline + fallback.
CRF vs bitrate?	CRF=constant quality, bitrate varies with content; +2 CRF ~=-25% bits. We control quality, not size.
Preset effect?	Same CRF, slower preset = smarter motion search = fewer bits. We use veryfast (latency) vs medium (baseline).
Why is video seconds not ms?	Encode searches motion across millions of blocks per frame - inherently seconds; ms claims would exclude the encode.
Where is the AI?	Heuristic policy + feedback loop; no runtime neural net. Beats fixed on all samples; CNN/BO is roadmap with research surveyed.
Failure handling?	Quality floor rejects bad output; exceptions propagate (no fake metrics); NEEDS-REVIEW when unevaluable; originals never overwritten.
PNG transparency?	Known limitation: converted to RGB, alpha dropped. Say it before they ask; fix = keep alpha via WebP lossless/palette path.
Scale the system?	Stateless FastAPI + ffmpeg workers, job queue (Celery/Redis), object storage, Postgres; GPU nodes for neural codecs later.

8. Edge Cases & How Code Handles Them

Edge case	Behaviour
Tiny file (adds header overhead)	Reported honestly - even negative saving; verdict MARGINAL; never hidden
PNG with transparency	Alpha dropped (RGB convert) - documented limitation
Corrupt/unsupported file	PIL/ffmpeg raises; API returns error, nothing fabricated
Huge image (>12MP)	auto downscale 0.75 (0.85 if >4MP) before encode
Short video (< sample timestamps)	skipped timestamps continue to next sample; total failure raises honestly
Silent video / no audio	AAC flags harmless; ffmpeg maps if present
Fixed beats AI on some file	possible - report shows both; retry floor guarantees no quality regression
Concurrent uploads	uuid filenames prevent collision; sqlite serializes logs
Stale UI after update	hard refresh (JS cached, seen as HTTP 304)

9. File Map (36 files)

Core: src/pipeline.py, selector.py, evaluator.py, analyzer.py, backend/main.py, src/scoring.py. Support: src/compressor.py, src/db.py, frontend/app.js. Evidence: scripts/benchmark.py, tests/test_pipeline.py. Docs: README, PRD, docs/*. Scaffolding (lowest priority): src/__init__.py, runs/.gitkeep, .gitignore, scripts/init_repo.ps1.